
LAPORAN PRAKTIKUM INTERNET OF THINGS

Praktikum 1



Oleh:

NAMA : Haikal Ahnaf Yahya

NPM : 25782098

KELAS : TRI – 3D

Dosen:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

**TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG**

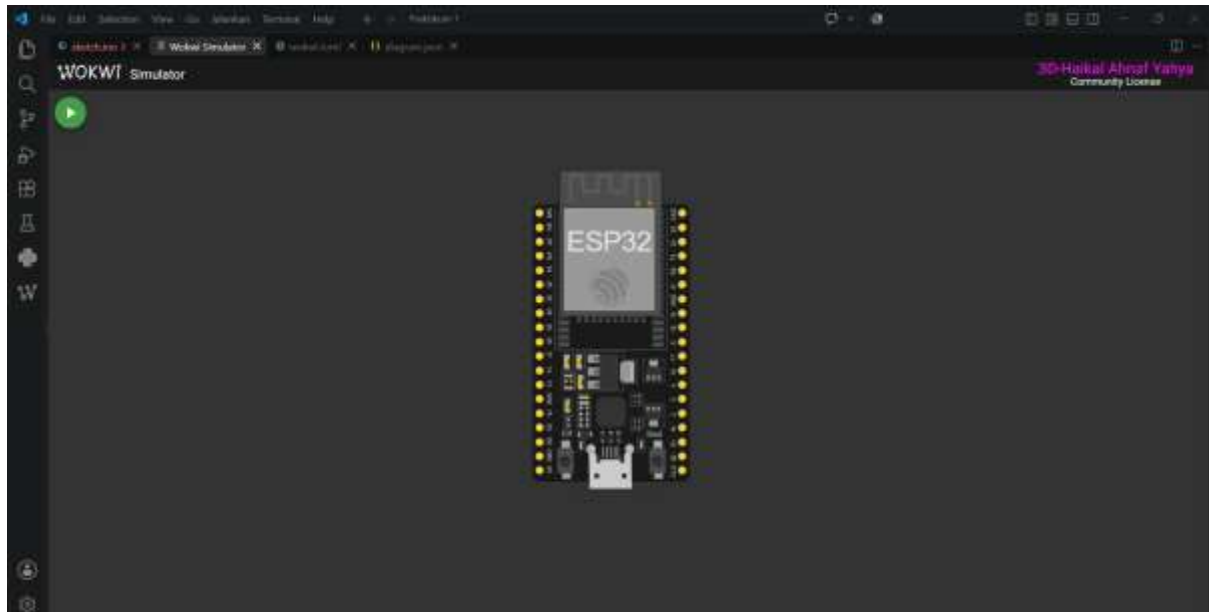
2025/2026

1.5.PRAKTIKUM

1.5.1 Praktikum 1: *Digital Output* (Menyalakan LED)

Langkah Kerja:

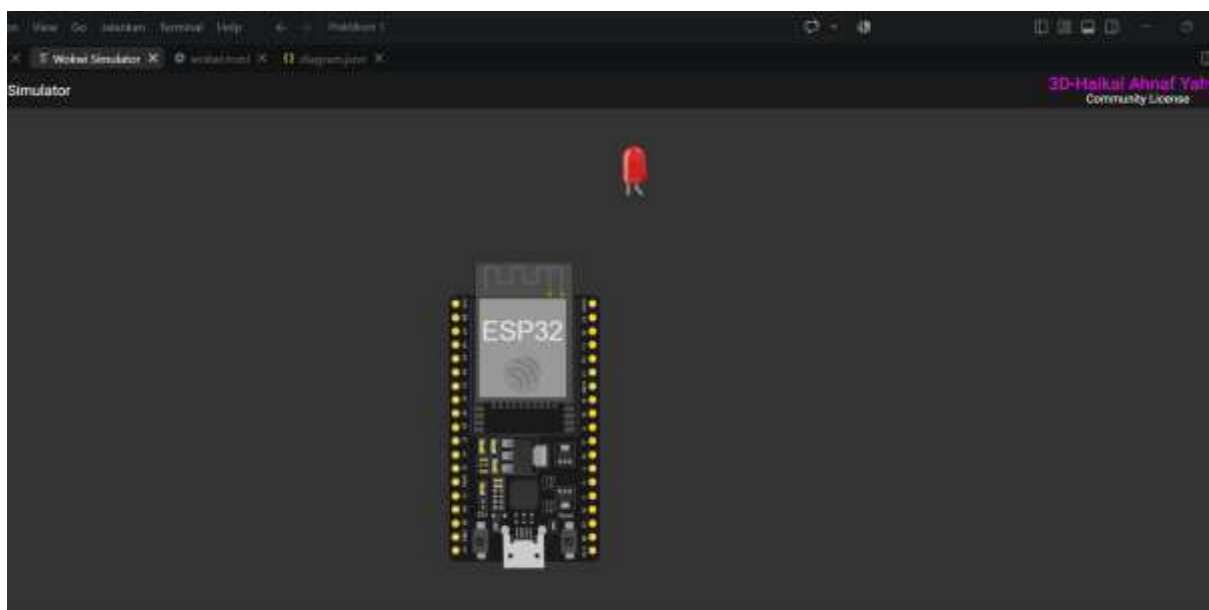
1. Pasang NodeMCU pada breadboard.



Di Wokwi, kita memang enggak bisa pasang NodeMCU di atas breadboard secara visual seperti praktikum di laboratorium fisik.

Soalnya, Wokwi didesain langsung secara skematik sederhana. Jadi, semua komponen—mulai dari papan ESP32/NodeMCU, LED, sampai resistor—langsung ditaruh berdampingan di area kerja virtual tanpa butuh tumpatan breadboard.

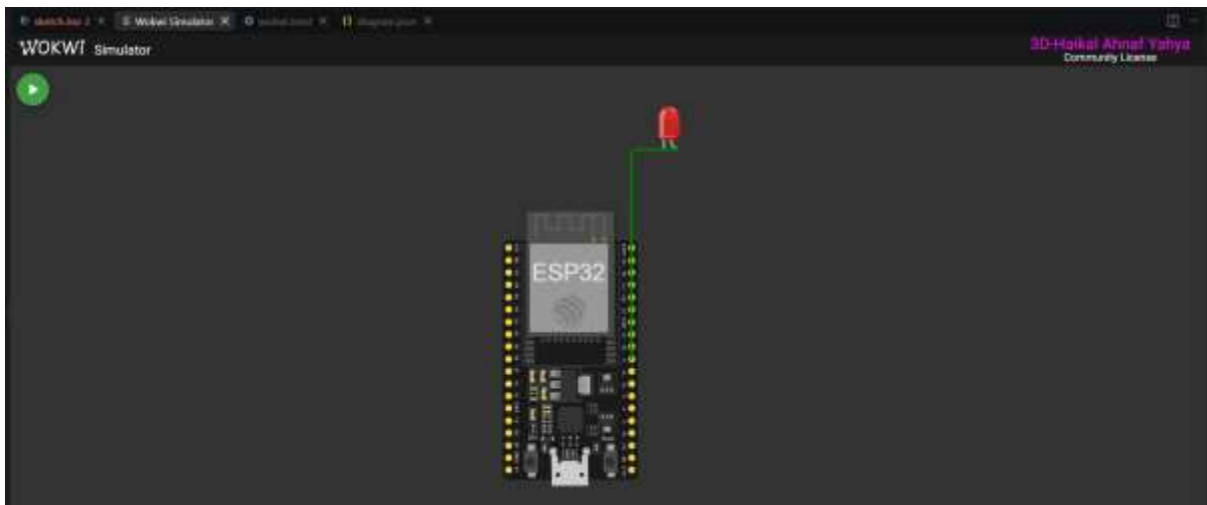
2. Pasang LED pada baris kosong di breadboard.:



Sama seperti LED fisik di laboratorium, LED virtual di Wokwi juga punya dua kaki dengan polaritas yang berbeda:

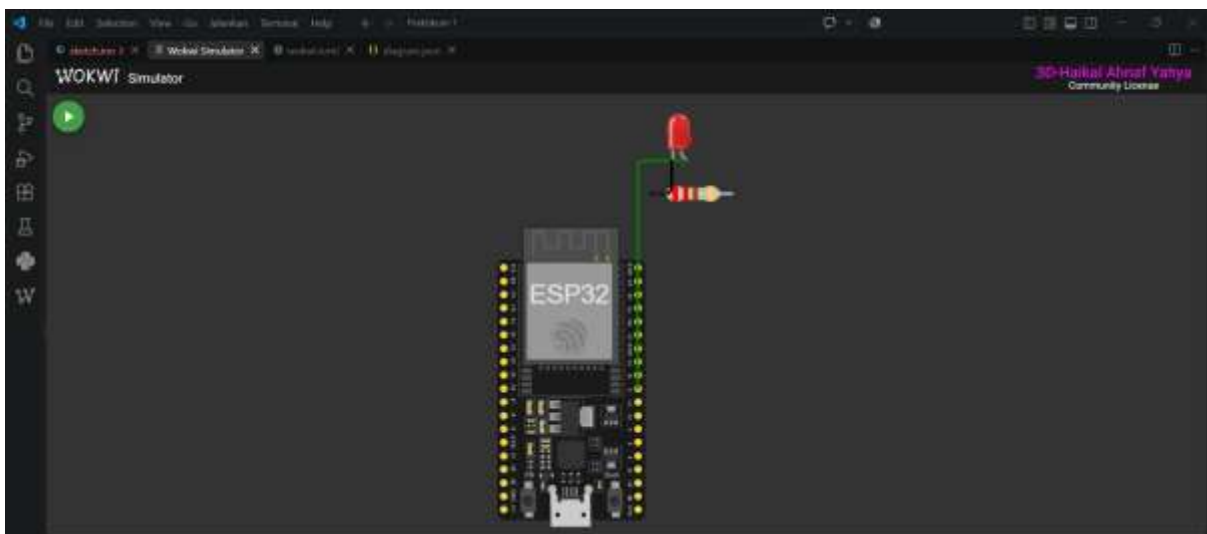
- Anoda (Positif / A): Pada tampilan Wokwi, kaki ini digambarkan sedikit membengkok ke luar (kaki yang lebih panjang). Kaki ini yang nanti akan disambungkan ke Pin Digital Output (GPIO 5).
- Katoda (Negatif / C): Pada tampilan Wokwi, kaki ini digambarkan lurus tegak (kaki yang lebih pendek). Kaki ini yang nanti akan disambungkan ke Resistor / GND.

3. Hubungkan pin D1 (GPIO 5) dari NodeMCU ke kaki Anoda LED menggunakan kabel jumper.



Kabel hijau ini berfungsi sebagai jalur pembawa sinyal listrik digital. Ketika program mengirim instruksi HIGH, pin GPIO 5 akan mengeluarkan tegangan 3,3V yang mengalir lewat kabel hijau ini menuju Anoda untuk menyalakan LED. Saat ini rangkaian belum aktif sepenuhnya karena kaki Katoda LED belum terhubung ke resistor dan Ground (GND).

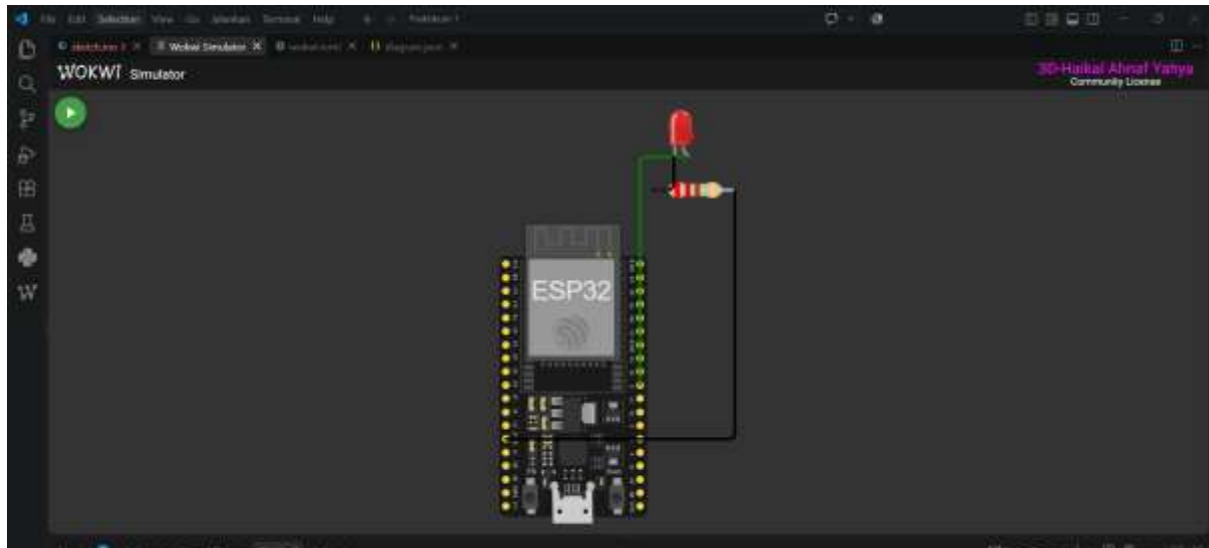
4. Hubungkan kaki Katoda LED ke salah satu ujung Resistor 220 Ohm.



Terlihat komponen Resistor 220 Ohm (dengan gelang warna merah-merah-cokelat) berhasil ditambahkan di bawah LED. Kaki Katoda LED (kaki kanan/negatif) terhubung ke salah satu

terminal resistor menggunakan kabel jumper berwarna hitam. Sambungan ini berfungsi menyalurkan arus listrik yang keluar dari LED menuju resistor untuk dibatasi besaran arusnya. Namun, arus listrik belum bisa mengalir karena sirkuitnya belum tertutup penuh. Jalur ini masih membutuhkan satu sambungan kabel lagi dari ujung resistor yang bebas menuju pin GND pada papan ESP32.

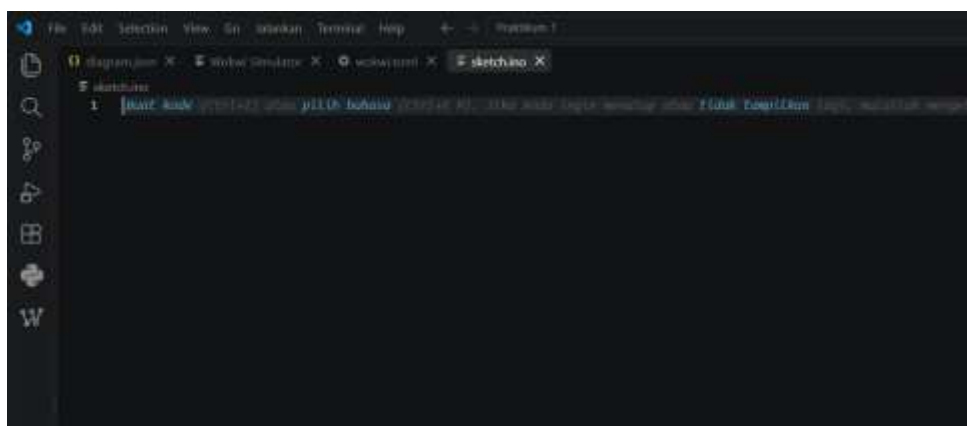
5. Hubungkan ujung Resistor lainnya ke jalur GND pada NodeMCU.



Terlihat ujung resistor lainnya sudah terhubung ke pin GND (Ground) pada papan ESP32 menggunakan kabel jumper berwarna hitam. Dengan terhubungnya kabel ini, seluruh rangkaian hardware sudah menjadi sirkuit tertutup (closed circuit). Arus listrik dapat mengalir sempurna dari pin GPIO 5 → Anoda LED → Katoda LED → Resistor 220 Ohm → Pin GND. Resistor di sini berfungsi sebagai pembatas arus agar LED tidak teraliri tegangan berlebih yang dapat merusaknya.

Langkah Kerja Pemrograman:

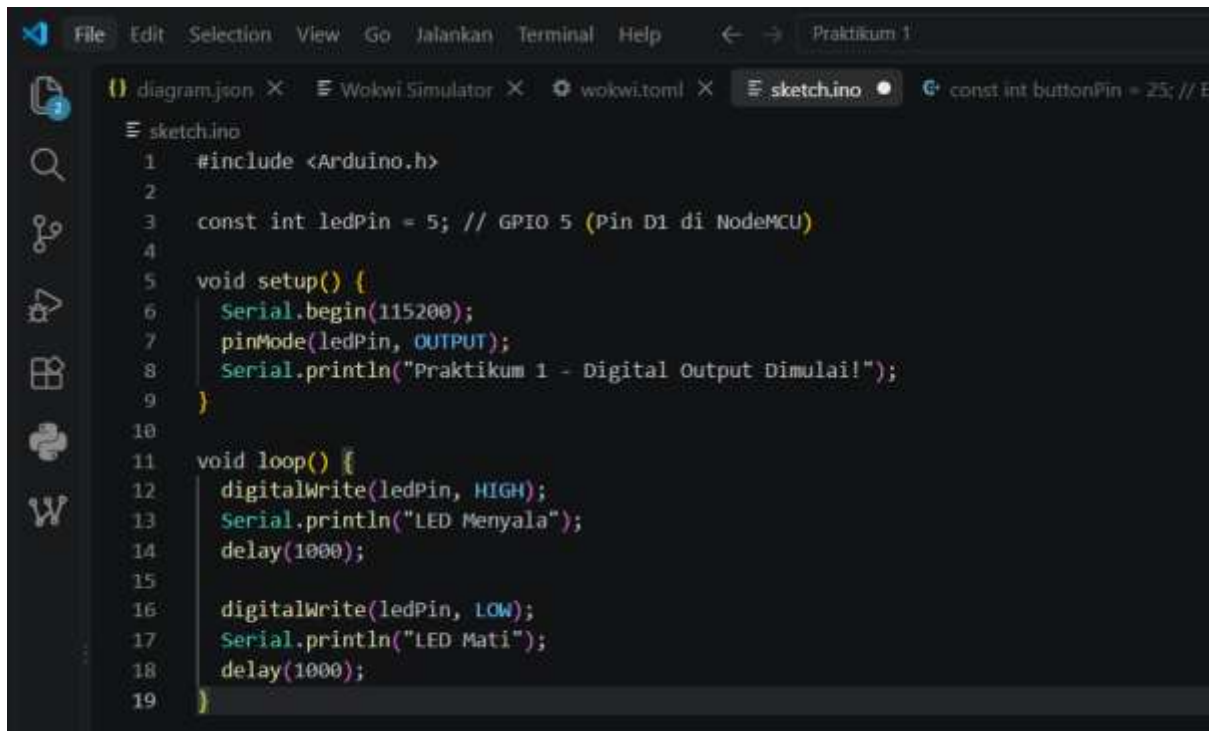
1. Buka Arduino IDE, klik File > New Sketch.



Teks pudar (placeholder) yang muncul di baris pertama merupakan petunjuk bawaan dari editor VS Code yang menandakan bahwa file ini siap untuk diisi. Lembar kerja kosong ini setara dengan tampilan awal saat membuat proyek baru di Arduino IDE (File > New Sketch), yang

nantinya akan diisi dengan logika program C++ untuk mengendalikan blinking LED pada ESP32.

2. Tuliskan kode program berikut:

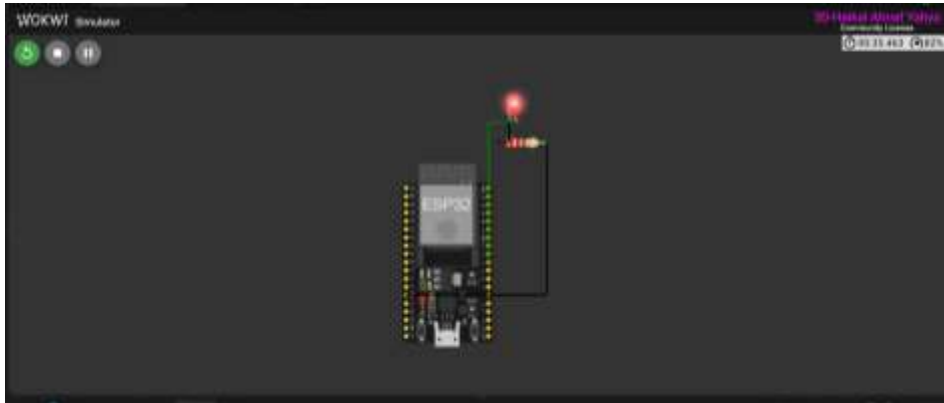
A screenshot of a code editor window titled 'Praktikum 1'. The editor shows a file named 'sketch.ino' with the following C++ code:

```
1  #include <Arduino.h>
2
3  const int ledPin = 5; // GPIO 5 (Pin D1 di NodeMCU)
4
5  void setup() {
6      Serial.begin(115200);
7      pinMode(ledPin, OUTPUT);
8      Serial.println("Praktikum 1 - Digital Output Dimulai!");
9  }
10
11 void loop() {
12     digitalWrite(ledPin, HIGH);
13     Serial.println("LED Menyala");
14     delay(1000);
15
16     digitalWrite(ledPin, LOW);
17     Serial.println("LED Mati");
18     delay(1000);
19 }
```

file sketch.ino berisi kode program C++ untuk membaca input push button dan mengontrol LED.

- `const int buttonPin = 4;`: Menentukan bahwa push button terhubung ke pin GPIO 4 (setara pin D2 pada NodeMCU).
- `const int ledPin = 5;`: Menentukan bahwa LED terhubung ke pin GPIO 5 (setara pin D1 pada NodeMCU).
- `int buttonState = 0;`: Menyimpan status pembacaan tombol (0/LOW atau 1/HIGH).
- `void setup()`: Berjalan satu kali saat sistem menyala, berfungsi mengaktifkan komunikasi serial dengan `Serial.begin(115200)`, serta mengatur `buttonPin` sebagai INPUT dan `ledPin` sebagai OUTPUT.
- `void loop()`: Berjalan berulang terus-menerus. Menggunakan `digitalRead(buttonPin)` untuk mengecek kondisi tombol; jika bernilai HIGH, LED dinyalakan dengan `digitalWrite(ledPin, HIGH)` dan mengirim teks ke Serial Monitor, sedangkan jika LOW, LED dipadamkan lewat `digitalWrite(ledPin, LOW)`.

3. Menjalankan Simulasi di VS Code



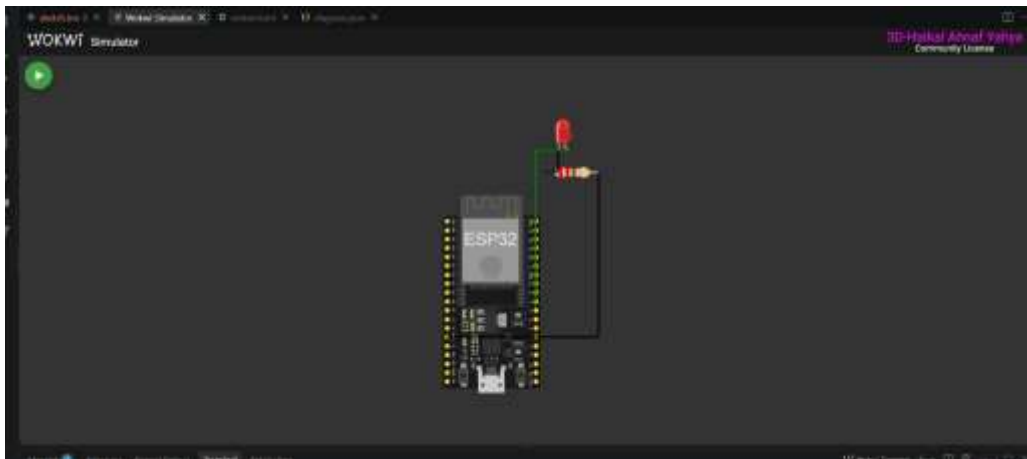
Simulasi sudah berhasil berjalan secara aktif:

- Kondisi Hardware Virtual: LED merah terlihat menyala terang (efek pendaran merah di sekitar LED), dan indikator daya pada papan ESP32 juga aktif.
- Alur Kerja: Sinyal HIGH dari GPIO 5 berhasil mengirimkan arus listrik melewati kabel hijau, menyalakan LED, lalu dibatasi oleh resistor 220 Ohm sebelum menuju ke Ground (GND). LED ini akan berkedip (*blinking*) secara otomatis setiap 1 detik sekali sesuai dengan interval delay(1000) pada kode program.

1.5.2 Praktikum 2: Digital Input (Membaca Push Button)

Langkah Kerja Rangkaian:

1. Biarkan rangkaian LED pada pin D1 dari Praktikum 1 tetap terpasang.

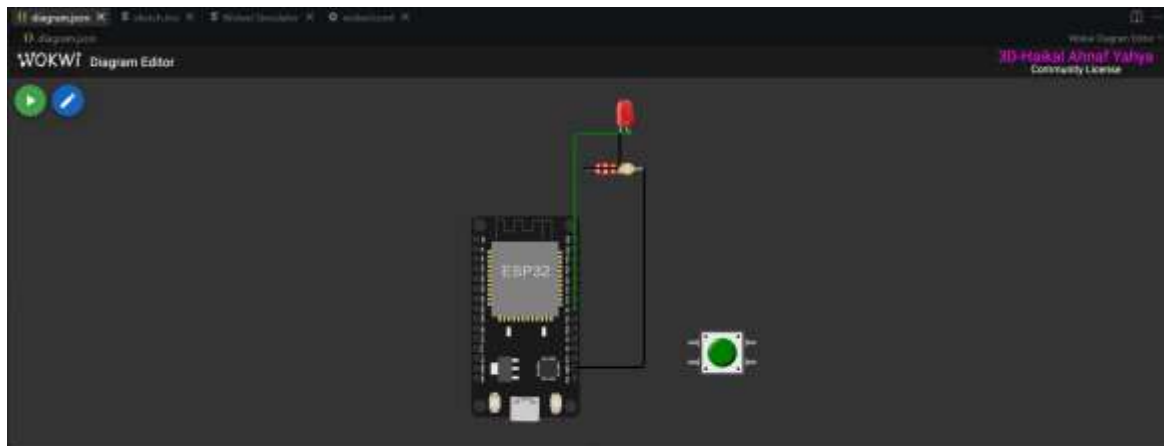


Seluruh rangkaian dari Praktikum 1 masih terpasang dengan lengkap dan rapi di area kerja Wokwi:

- Sinyal Output: Pin GPIO 5 (D1) terhubung ke Anoda LED lewat kabel hijau.
- Jalur Ground & Resistor: Katoda LED terhubung ke Resistor 220 Ohm, lalu disambungkan ke pin GND lewat kabel hitam.

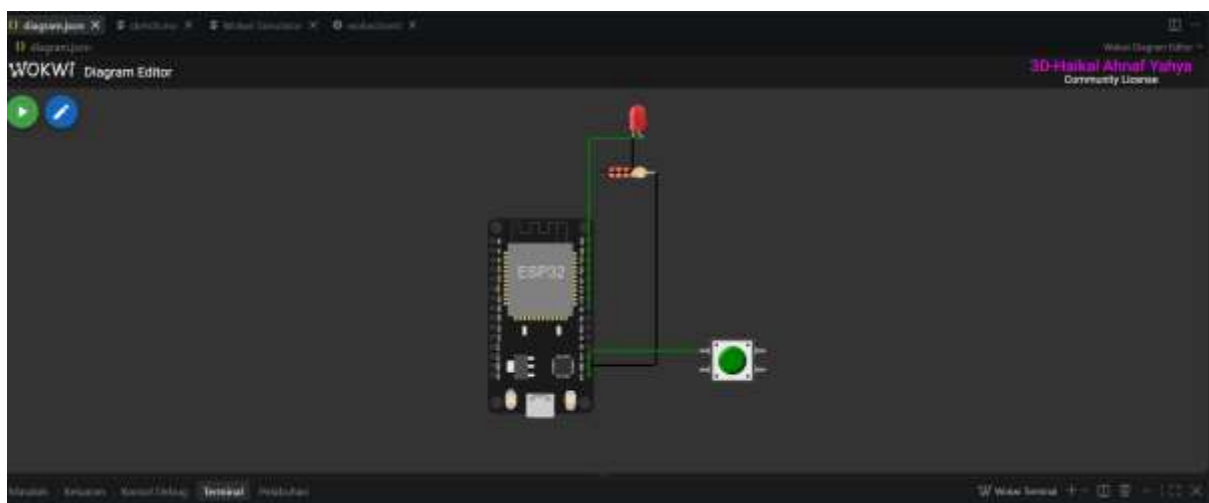
Rangkaian output ini dipertahankan sebagai indikator visual (LED) yang nantinya akan menyala saat tombol pada Praktikum 2 ditekan.

2. Pasang sebuah push button pada breadboard, melintasi garis pemisah/parit tengah.



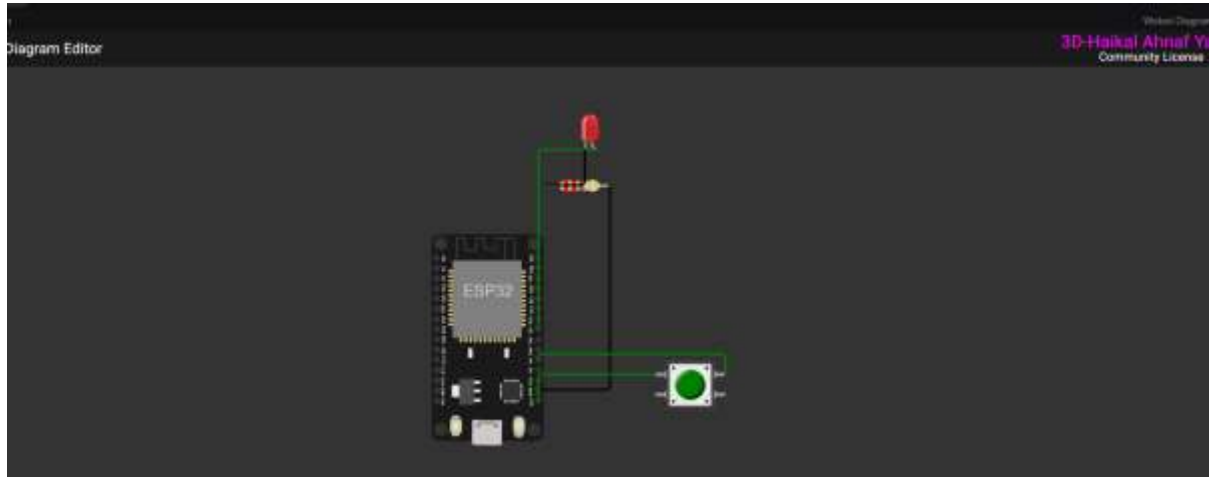
Push Button berwarna merah sudah berhasil ditambahkan di sebelah kanan papan ESP32. Tombol ini memiliki empat kaki terminal (dua di sebelah kiri dan dua di sebelah kanan). Pada tahap ini, tombol baru hanya diletakkan di area kerja virtual Wokwi dan belum terhubung ke kabel atau pin mana pun, sehingga belum bisa menyalurkan atau mendeteksi sinyal listrik.

3. Hubungkan kaki pertama *push button* ke pin 3V3 (3.3V) pada NodeMCU.



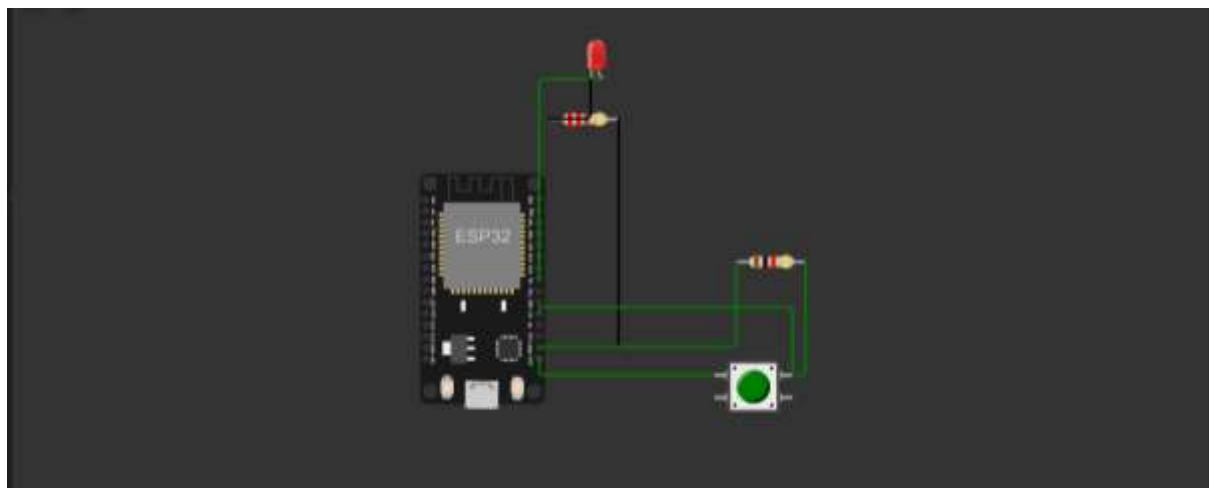
Kaki kiri atas push button sekarang sudah terhubung ke pin 3V3 (tegangan positif 3,3 Volt) pada papan ESP32 menggunakan kabel merah. Jalur ini berfungsi untuk menyuplai arus listrik positif ke salah satu sisi tombol. Saat tombol ditekan nantinya, tegangan 3,3V ini akan mengalir menyeberang ke kaki tombol di sebelah kanan.

4. Hubungkan kaki kedua push button ke pin D2 (GPIO 4) NodeMCU.



Kaki kanan atas push button sekarang sudah terhubung ke pin GPIO 4 pada papan ESP32 menggunakan kabel hijau. Jalur ini berfungsi sebagai saluran sinyal masuk (digital input) yang bertugas membaca kondisi status tombol. Saat tombol ditekan, sinyal tegangan 3,3 Volt akan diteruskan melalui kabel biru ini ke mikrokontroler sehingga terbaca sebagai kondisi HIGH.

5. Pada kaki kedua yang sama, pasang sebuah Resistor 10k Ohm, lalu hubungkan ujung lain dari resistor tersebut ke jalur GND NodeMCU.

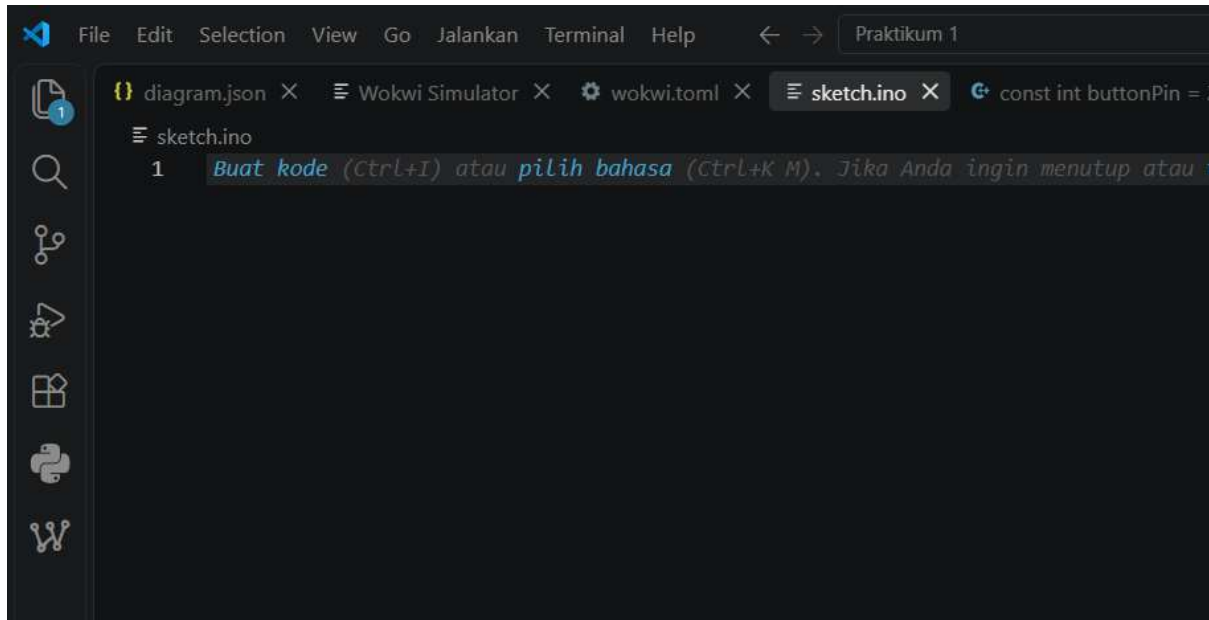


Kaki kanan atas push button (yang terhubung ke pin GPIO 4) sekarang disambungkan ke salah satu kaki resistor 10k Ohm. Ujung kaki resistor lainnya terhubung ke pin GND pada ESP32 menggunakan kabel hijau. Jalur ini berfungsi sebagai resistor pull-down yang menarik tegangan pada pin GPIO 4 menuju 0 Volt (ground) ketika tombol tidak ditekan, sehingga memastikan status pembacaan input berada dalam kondisi stabil LOW dan terhindar dari sinyal

liar (floating signal). Saat push button ditekan, arus 3.3V dari pin 3V3 akan mengalir ke pin GPIO 4 sehingga pembacaan logika berubah secara pasti menjadi HIGH.

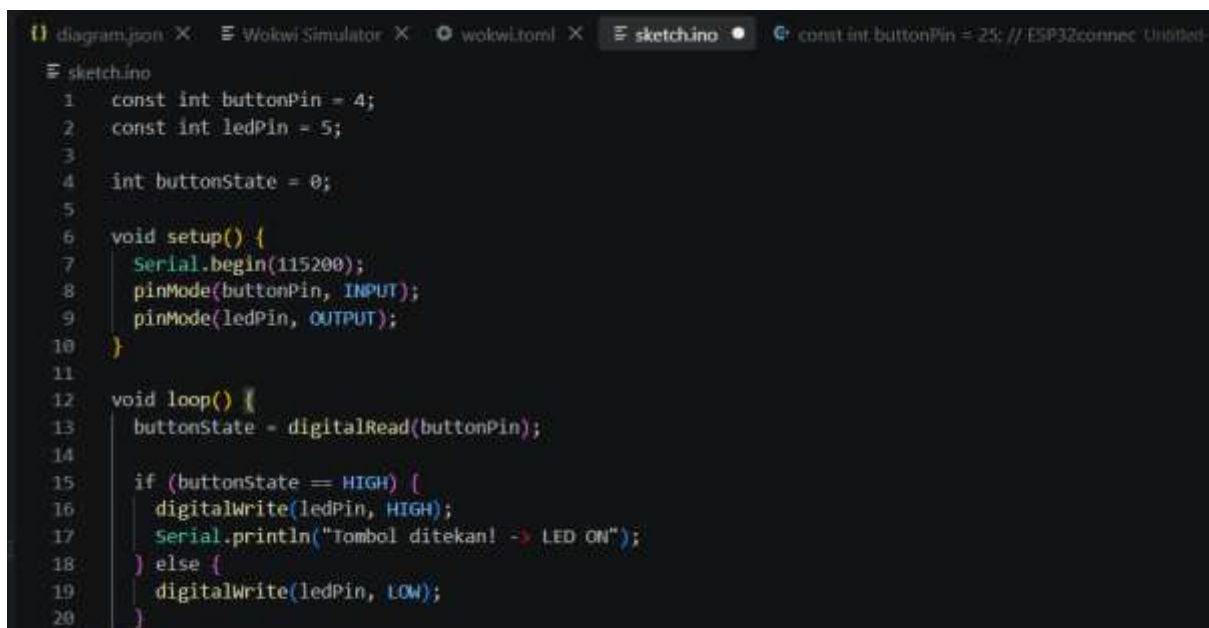
Langkah Kerja Pemrograman:

1. Buat berkas baru (New Sketch) pada Arduino IDE.



Editor berkas **sketch.ino** di VS Code masih dalam keadaan kosong. Berkas ini berfungsi sebagai tempat untuk menuliskan seluruh instruksi program utama C++ yang nantinya akan dieksekusi oleh mikrokontroler ESP32.

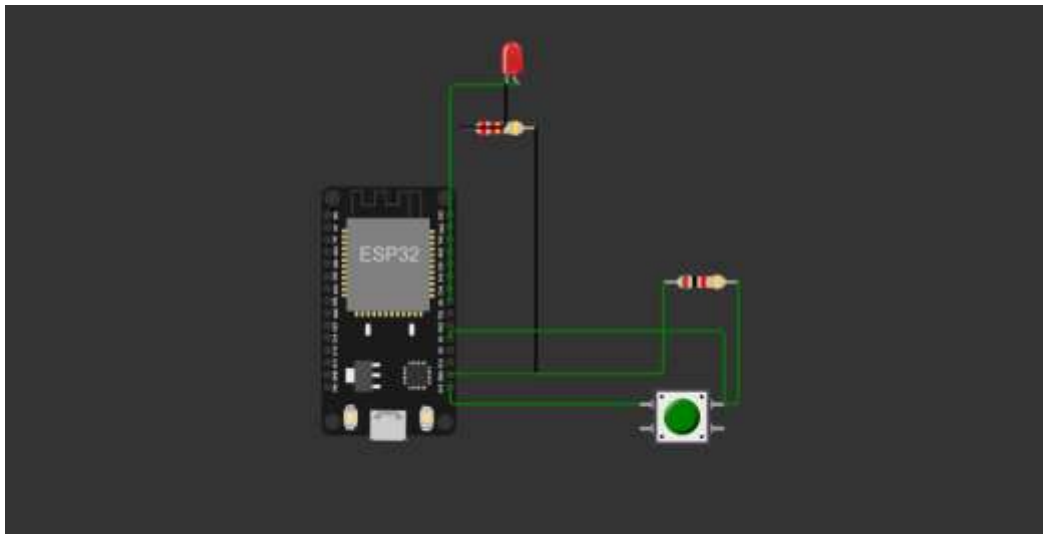
2. Tuliskan kode program berikut:



Kode di atas mendefinisikan buttonPin (GPIO 4) sebagai input tombol dan ledPin (GPIO 5) sebagai output LED, serta variabel buttonState untuk menyimpan statusnya. Pada setup(), komunikasi Serial dibuka pada 115200 baud, lalu mode pin diatur sesuai fungsinya. Pada loop(), perintah digitalRead() memeriksa kondisi tombol. Jika tombol ditekan (HIGH), LED menyala dan pesan dikirim ke Serial Monitor. Jika tombol dilepas (LOW), LED akan padam.

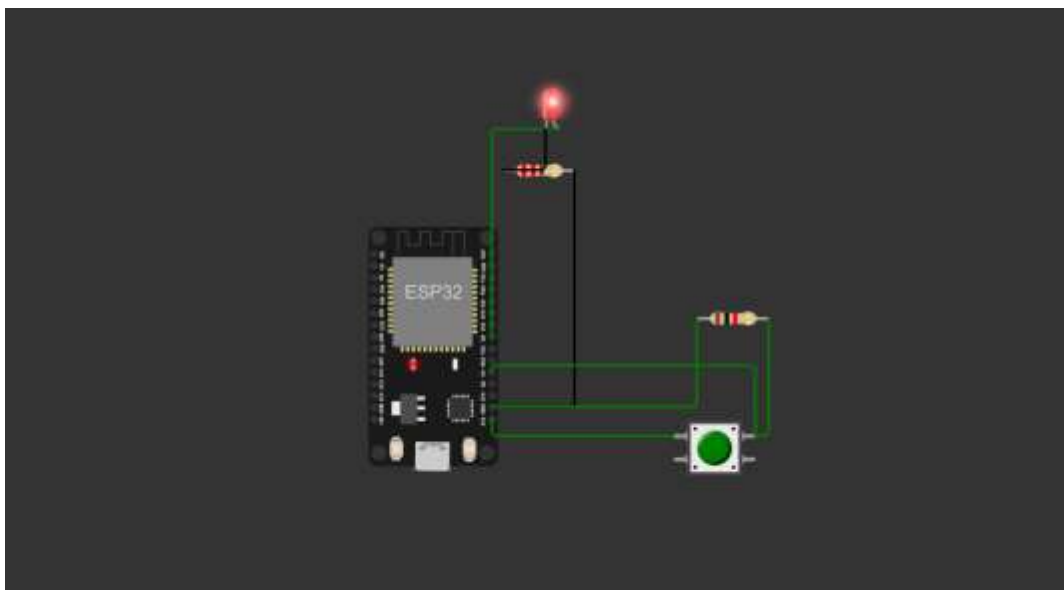
3. Uji fungsionalitasnya: Saat tombol ditekan dan ditahan, LED akan menyala terang. Saat tombol dilepaskan, LED langsung padam.

- Saat Tombol Tidak Dipencet (Dilepas)



Pada kondisi ini, aliran listrik dari pin 3V3 terputus di dalam push button. Pin GPIO 4 terhubung ke GND melalui resistor pull-down ($10k\Omega$), sehingga ESP32 membaca sinyal logika LOW (0V). Kode program menjalankan kondisi else, sehingga pin GPIO 5 berada pada tegangan 0V dan LED dalam keadaan padam (OFF).

- Saat Tombol Dipencet (Ditekan)



Saat tombol ditekan, kontak internal sakelar terhubung sehingga aliran arus dari pin 3V3 masuk ke pin GPIO 4. ESP32 membaca perubahan sinyal dari 0V menjadi HIGH (3,3V). Kondisi ini pada program terpenuhi, sehingga ESP32 mengirim sinyal HIGH ke pin GPIO 5 yang membuat LED menyala terang (ON) dan mengirimkan pesan teks ke Serial Monitor.

1.6 Latihan

1. (Pemahaman Logika Pin): Berdasarkan tabel Referensi Pinout di subbab 1.3.3, jelaskan apa yang mungkin terjadi jika Anda menggunakan logika rangkaian dari Praktikum 2 (Active-High / pull-down), namun menyambungkan tombol tersebut ke pin D8 (GPIO 15), lalu menekan tombol tersebut tepat pada detik di mana NodeMCU baru diberi daya?
 - Pin GPIO 15 (D8) pada ESP8266/NodeMCU merupakan salah satu strapping pin yang menentukan boot mode saat papan pertama kali diberi daya (power-up). Syarat wajib agar NodeMCU dapat masuk ke mode normal boot dari memori flash adalah GPIO 15 harus berada dalam kondisi LOW (0V) saat proses booting.
 - Yang terjadi jika tombol ditekan saat diberi daya: Rangkaian Active-High/pull-down akan mengalirkan tegangan 3,3V (HIGH) langsung ke GPIO 15 saat tombol ditekan. Jika tombol ditekan tepat saat diberi daya, NodeMCU akan gagal masuk ke bootloader internal dan berpotensi masuk ke boot mode yang salah (seperti SD Card boot atau UART download mode tergantung kombinasi pin lainnya). Akibatnya, NodeMCU mengalami gangguan booting (stuck/gagal menyala) dan program tidak akan berjalan.
2. (Implementasi Kode): Modifikasi kode pada Praktikum 1 agar LED berkedip secara tidak simetris, yaitu menyala sangat cepat selama 200 ms, kemudian mati agak lama selama 800 ms. Tuliskan dua baris fungsi delay() yang harus Anda ubah.

Dua baris fungsi delay() yang harus diubah pada loop() adalah:

```
void loop() {  
    digitalWrite(ledPin, HIGH); // LED menyala  
    delay(200);                // Ubah baris ini (200 ms)  
  
    digitalWrite(ledPin, LOW); // LED mati  
    delay(800);                // Ubah baris ini (800 ms)}
```

3. (Analisis Sirkuit): Berdasarkan hukum aliran arus listrik searah, apa yang terjadi jika pada Praktikum 1 Anda memasang kaki LED secara terbalik (Anoda ke GND, Katoda ke D1)? Apakah program akan mengalami error kompilasi? Mengapa LED tidak menyala?
 - Tidak. Program akan berhasil di-compile dan di-upload tanpa pesan kesalahan. Error kompilasi (syntax error) hanya terjadi jika ada kesalahan pada penulisan kode C++, bukan karena kesalahan pemasangan kabel fisik.
 - Berdasarkan sifat arus searah pada dioda, LED hanya dapat mengalirkan arus dari Anoda (positif) ke Katoda (negatif) (forward bias). Jika Anoda dihubungkan ke GND (0V) dan Katoda ke D1 (yang mengeluarkan 3,3V saat HIGH), LED berada dalam kondisi tersumbat (reverse bias). Tegangan terbalik ini menahan aliran listrik sehingga arus tidak dapat mengalir dan LED tidak akan menyala.

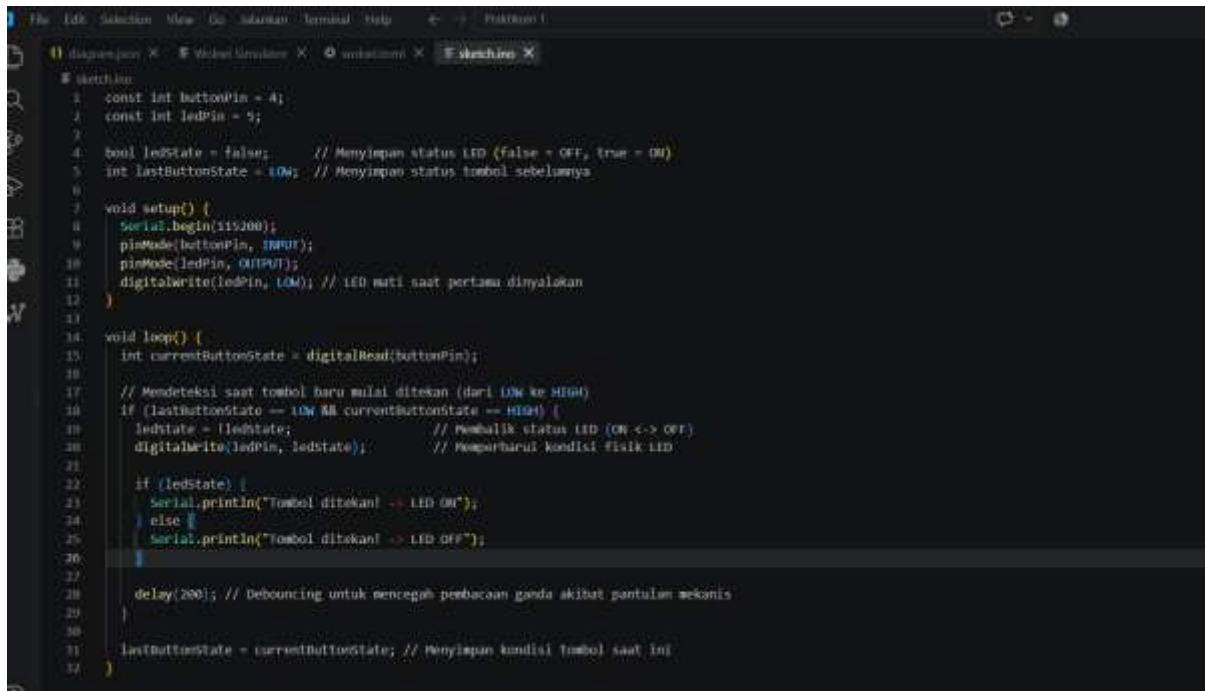
4. (Pengembangan Algoritma): Ubah blok kondisi if-else pada Praktikum 2 sehingga perilakunya terbalik: "Saat tombol TIDAK ditekan, LED menyala. Saat tombol DITEKAN, LED justru mati."

- Untuk membalik perilaku sakelar pada logika Active-High (tombol TIDAK ditekan = LED ON, tombol DITEKAN = LED OFF), blok if-else disesuaikan menjadi seperti berikut:

```
if (buttonState == HIGH) {  
    digitalWrite(ledPin, LOW); // Saat tombol ditekan, LED justru mati  
    Serial.println("Tombol ditekan! -> LED OFF");  
} else {  
    digitalWrite(ledPin, HIGH); // Saat tombol TIDAK ditekan, LED menyala  
    Serial.println("Tombol dilepas -> LED ON");  
}
```

1.7 Tugas

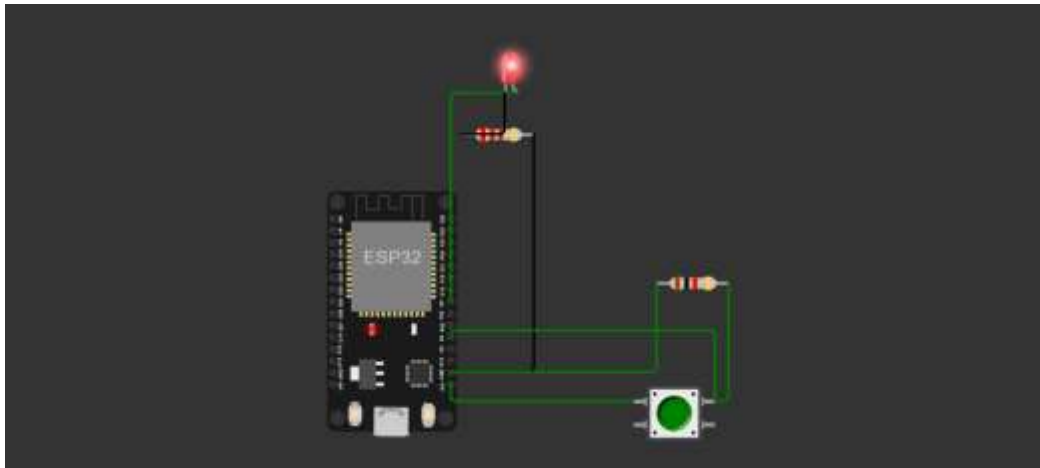
1. Modifikasi kode program sehingga tombol berfungsi layaknya sakelar lampu ruangan dan Gunakan variabel penyimpanan status (misalnya tipe boolean atau int) untuk melacak apakah lampu saat ini sedang ON atau OFF. Anda juga membutuhkan instruksi delay() singkat (sekitar 200 milidetik) tepat setelah mendeteksi tombol ditekan, untuk mengatasi fenomena pantulan mekanis (*debouncing* pada tombol fisik).



```
# sketch.ino  
1  const int buttonPin = 4;  
2  const int ledPin = 5;  
3  
4  bool ledState = false; // Menyimpan status LED (false = OFF, true = ON)  
5  int lastButtonState = LOW; // Menyimpan status tombol sebelumnya  
6  
7  void setup() {  
8      Serial.begin(115200);  
9      pinMode(buttonPin, INPUT);  
10     pinMode(ledPin, OUTPUT);  
11     digitalWrite(ledPin, LOW); // LED mati saat pertama dinyalakan  
12 }  
13  
14 void loop() {  
15     int currentButtonState = digitalRead(buttonPin);  
16  
17     // Mendeteksi saat tombol baru mulai ditekan (dari LOW ke HIGH)  
18     if (lastButtonState == LOW && currentButtonState == HIGH) {  
19         ledState = !ledState; // Membalik status LED (ON <-> OFF)  
20         digitalWrite(ledPin, ledState); // Memperbarui kondisi fisik LED  
21  
22         if (ledState) {  
23             Serial.println("Tombol ditekan! -> LED ON");  
24         } else {  
25             Serial.println("Tombol ditekan! -> LED OFF");  
26         }  
27  
28         delay(200); // Debouncing untuk mencegah pembacaan ganda akibat pantulan mekanis  
29     }  
30  
31     lastButtonState = currentButtonState; // Menyimpan kondisi tombol saat ini  
32 }
```

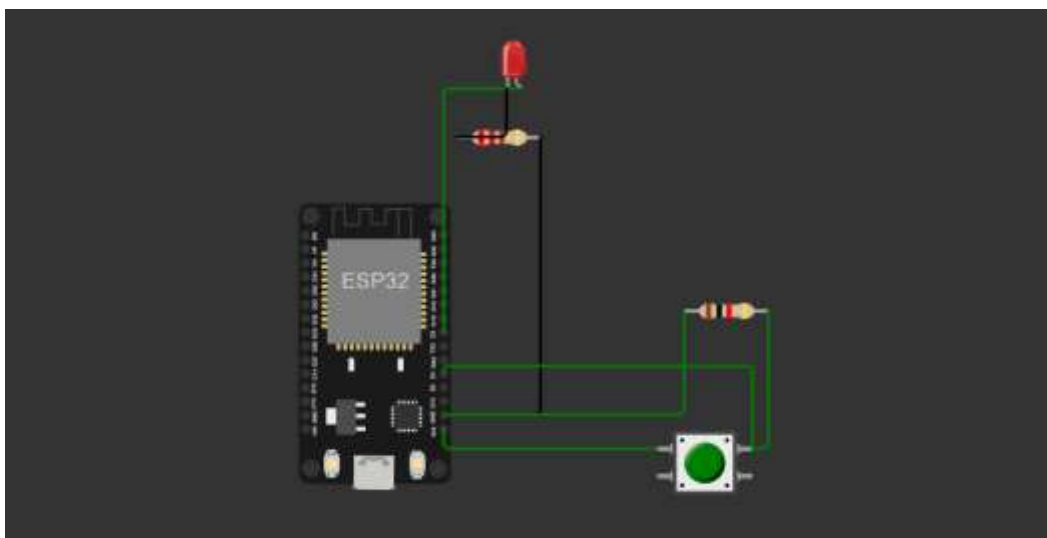
- ledState: Variabel bertipe boolean untuk menyimpan kondisi LED saat ini (mati atau menyala).
- Deteksi Tekanan (lastButtonState == LOW && currentButtonState == HIGH): Program hanya merespons pada momen awal tombol ditekan, bukan saat ditekan secara terus-menerus.

- `ledState = !ledState`: Membalik status LED (toggle). Jika sebelumnya mati (false) akan menjadi menyala (true), dan sebaliknya.
 - `delay(200)`: Berfungsi sebagai debouncing untuk menstabilkan sinyal fisik tombol agar satu kali tekanan tidak terbaca berulang kali.
- Jika tombol ditekan 1 kali lalu dilepas, LED menyala dan tetap menyala.



Pada hasil simulasi di atas, saat tombol ditekan 1 kali, sinyal pada pin GPIO 4 berubah dari LOW ke HIGH. Perubahan kondisi ini terdeteksi oleh program (kondisi if), sehingga status variabel `ledState` dibalik dari false menjadi true. Program kemudian mengirim sinyal HIGH ke pin GPIO 5 yang menyebabkan LED merah menyala dan tetap menyala. Instruksi `delay(200)` dijalankan untuk mencegah efek debouncing (pantulan sinyal ganda).

- Jika tombol ditekan 1 kali lagi lalu dilepas, LED mati dan tetap mati.



Ketika tombol ditekan 1 kali lagi lalu dilepas, sinyal pada pin GPIO 4 kembali terdeteksi berubah dari LOW ke HIGH. Program kembali mengeksekusi logika if dan membalik nilai variabel `ledState` dari true menjadi false. Selanjutnya, program mengirim sinyal LOW ke pin GPIO 5 sehingga aliran arus terputus dan LED merah mati serta tetap mati hingga tombol ditekan kembali.